



ORACLE INTERVIEW EXPERIENCE

Oracle Financial Services Software (OFSS) is a wholly owned subsidiary of Oracle Corporation that delivers enterprise-class IT solutions to banks and financial institutions worldwide. Headquartered in Mumbai, India, OFSS provides a comprehensive suite of products for core banking, digital banking, risk and compliance management, analytics, and payments.

The company combines Oracle's global technology leadership with deep financial-services domain expertise, helping institutions modernize their infrastructure, improve customer experience, and meet regulatory requirements. With clients in more than 145 countries, OFSS offers both product development and IT services/consulting, supporting banks of all sizes in their digital-transformation journeys.

Role Offered: Associate Application Developer (OFSS)

Selection Process:

1. Online Test
2. PPT @CUIC
2. Technical Interview
3. Techno-Managerial

Online Test:

The online test was conducted on the **HackerRank platform** and consisted of **3 sections**:

1. Technical Questions

- Covered **Core CS Fundamentals**: Computer Networks (CN), OOPS, DBMS, and Operating Systems (OS).
- Level: **Easy to Medium**.

2. Aptitude & English Knowledge

- Focused on **basic aptitude skills** and **English comprehension**.

3. Coding Questions (2 LeetCode Medium-level Problems)

Problem 1: Find Special Nodes in Tree

Problem Statement:

You are given a tree with `tree_nodes` nodes. A node is considered **special** if it lies at the **end of the tree diameter**.

- The **diameter** of a tree is defined as the longest path between any two nodes, measured in edges.
- Return a binary list `res[]` of size `tree_nodes` where:
 $\text{res}[i] = 1$ if the $(i+1)$ -th node is special
 $\text{res}[i] = 0$ otherwise

Example:

Input:

`tree_nodes = 7`

`tree_from = [1, 2, 3, 3, 1, 1]`

`tree_to = [2, 3, 4, 5, 6, 7]`

Output:

`[0, 0, 0, 1, 1, 1, 1]`

Problem 2: Detect High Load Windows

Problem Statement:

- You are given a list of **system loads** recorded each minute, a **window size**, and a **threshold**.
- For each minute, compute the average load of the last `windowSize` minutes (including the current minute).
- Return all minutes where this average is **strictly greater than the threshold**.

Example:

Input:

`loads = [8, 14, 5, 17]`

`windowSize = 2`

`threshold = 10`

Output:

`[1, 3]`

18 students were called for interviews @CUIC

Technical Round:

1. Introduction

2. Project Discussion (ML & NLP Project + Full-Stack Project)

3. Questions asked:

- **What is Cosine Similarity with examples?**

→ Explained with an example from my project:

It measures the **cosine of the angle between two vectors**.

Used in NLP to measure similarity between text embeddings.

Gave example with some sentences and finding similarity

- **Explain how TextRank Algorithm works.**

→ I explained graph-based ranking for NLP tasks like **extractive summarization**.

Represent sentences as nodes.

Edges = similarity scores between sentences.

Apply PageRank to rank sentences → select top sentences as summary.

- **What is BERT? Why Pegasus?**

→ BERT is a **transformer-based bidirectional encoder** pre-trained on Masked Language Modeling and Next Sentence Prediction.

Pegasus is better for **abstractive summarization**, trained on massive news & web data, with **gap-sentence generation** objective.

They asked on the architecture of these models

- **Preprocessing methods used? Why?**

Tokenization

Stop-word removal

Lemmatization

Lowercasing

- **Why did you choose this project? Use cases?**

- **Web Services flow of data with code (React & Express code snippets):**

→ Explained with **request-response cycle**:

Client makes a request (async/await used for non-blocking).

Server processes and sends JSON/XML response.

Example with **fetch, async, await** in frontend to handle asynchronous calls.

Code snippets were asked for the whole cycle

- **JWT – structure, need, and workflow (Code Snippets were asked):**

Header, Payload, Signature.

Workflow:

User logs in → token signed and issued.

Client stores token (localStorage).

On every request → token sent for validation.

Server verifies with secret key.

- **Difference on Authentication vs Authorization:**

Authentication → verifying **who** you are.

Authorization → verifying **what** you are allowed to do.

- **What is Open Authentication (OAuth):**

Standard protocol for third-party authentication (e.g., login with Google).

- **Why MongoDB for your project?**

→ Schema flexibility: post → comments → child comments → n nested comments → best represented as nested documents in NoSQL.

→ Avoids complex joins of SQL.

- **Why combination of SQL + NoSQL for multimedia apps?**

SQL: Structured, relational metadata.

NoSQL: Unstructured data (images, videos).

Explained Multimedia Database Management System

HTTP vs HTTPS – Why HTTPS is secure?

HTTPS uses **SSL/TLS encryption**, ensures **confidentiality, integrity, and authentication**.

- **Self-rating in CP:** I rated myself 8.5/10. Below was the question asked ..

You are asked to design a data model to represent a **large, dynamic class hierarchy** with the following properties. **Code implementation of the entire class structure and functions was asked.**

Root Class:

There is a single base class called Node.

Alphabetical Branching:

Each class can have up to 26 direct subclasses, one for each letter A through Z.

Each of those subclasses can themselves have A-Z subclasses, and so on to arbitrary depth

Inheritance Rules:

Multiple inheritance is allowed, but the design must prevent the “diamond problem” (no class should inherit from the same ancestor along two different paths). In other words, a class may inherit from more than one parent, but the resulting graph must remain a directed acyclic graph (DAG) with no converging cycles.

Dynamic Updates:

Classes (nodes) may be added, removed, or modified at any time.

The structure must handle these operations efficiently without downtime.

Class Metadata:

Each class can define its own set of methods, properties, and other metadata.

Required Operations

1. **Fast Retrieval**
Quickly locate a specific class by its full hierarchical path (e.g., Node → C → F → ...).
2. **Efficient Search**
Support lookups by name or prefix (e.g., “find all classes whose path starts with AB”).
Handle queries such as “find all subclasses that override method X.”
3. **Traversal and Batch Processing**
Enumerate or update all subclasses within any subtree or inheritance branch.
4. **Dynamic Maintenance**
Support **real-time insertion, deletion, and re-linking of inheritance relationships**

My Approach & Code was written in Python:

My solution for the system as a **Directed Acyclic Graph (DAG)**, where each class is a graph node. A node stores its name, its methods, links to its parents and children, and also a cached set of all ancestors for fast validation. To make class lookup efficient, we maintain a **Trie (prefix tree)** where each path like "A/B/C" maps directly to the corresponding class node. This gives fast lookups by path and also makes prefix-based searches easy.

To prevent the **diamond problem**, whenever we add a new inheritance link, we check two things:

1. No cycles are allowed (a class cannot become its own ancestor).
2. No ancestor overlap between parents—if two parents share a common ancestor, adding this link would create a diamond, so we reject it.
3. The structure supports **dynamic updates**:
 - Adding a class means creating a new node, inserting it into the trie, and linking it under a default root. Adding inheritance requires running the cycle and overlap checks, then updating ancestor sets. Removing a class just means unlinking it from its parents/children and updating descendants.
 - For **retrieval and search**, path lookups are done by following the trie, prefix-based searches are just trie traversals, and traversals like BFS/DFS can be run directly on the graph of nodes.

Graph (DAG) + Trie + Ancestor checks

Techno-Managerial Round:

1. Self-introduction
2. Pegasus Transformer – use cases, training data, architecture.
3. How companies maintain data security – I explained RAG (Retrieval Augmented Generation) + access controls.
4. About OFSS – main products (learned from PPT).
5. Software Design Principles – my approach to problem-solving.
6. Project explanations.
7. Knowledge rating in Java & Python (with follow-up coding questions).
8. Preference: AI or Development?
9. Basic ML Concepts – e.g., overfitting, embeddings.
10. How LLM's understand semantic of a sentence?
11. AI in Finance/Banking – explained fraud detection pipelines.

- 12.Spring in banking sectors – modular, secure, scalable.
- 13.Advantages of Spring Boot – auto-configuration, embedded servers, microservice-ready.
- 14.Web Services in Java – I wrote Java Servlet code for request/response.
- 15.Testing methods & bug fixing – unit testing, integration testing, regression. For user-side errors → replicate & trace logs.
- 16.Deadline handling – standard deadline scenario questions
- 17.Learning new languages/concepts – start with basics documentation → hands-on practice → build mini-projects.
- 18.Future of AI – discussed industry shift towards AI + Finance + Governance.
- 19.Time given to ask interviewer questions.

Final Thoughts

Overall, the experience tested end-to-end skills:

- Core CS fundamentals
- Coding & problem-solving
- System design thinking
- Project depth in ML, NLP & Full-Stack
- Managerial & communication aspects

Preparation Tips:

1. Understand Core CS Fundamentals Thoroughly

- Revise **OOPS, DBMS, Operating Systems, and Computer Networks.**
- Focus on concepts like **transactions, concurrency, deadlock handling, HTTP/HTTPS, authentication vs authorization**, etc.

2. Coding & Problem-Solving

- Practice **medium-level coding problems** on platforms like **LeetCode, HackerRank, and GeeksforGeeks.**
- Focus on **trees, graphs, sliding windows, dynamic programming, and design problems.**
- Learn to explain your **approach, time/space complexity**, and edge cases clearly.

3. System Design & Projects

- Be ready to **explain your projects in depth**, including your **technical choices, challenges, and trade-offs.**

- Prepare to **design data models and class hierarchies**.
- Understand **web services, API flow, and authentication mechanisms** like JWT.

4. Machine Learning & NLP Concepts

- Know **transformer-based models** (BERT, Pegasus) and **how they work**.
- Be able to explain **preprocessing techniques, embeddings**.

5. Programming Languages & Frameworks

- Brush up on **Java and Python**, including syntax, data structures, and libraries used in your projects.
- Be ready to write small snippets of **web service code** (servlets, async requests, REST APIs).

6. Behavioral & Managerial Questions

- Be prepared to discuss **deadline management, learning new concepts, debugging approaches, and teamwork**.
- Practice explaining **your approach to problem-solving and system design** clearly.

7. Mock Interviews & Explanation Skills

- Conduct **mock interviews** focusing on **explaining your reasoning clearly**.
- Be able to **justify technical choices** and explain **project workflows** concisely.

8. Time Management & Confidence

- Allocate time wisely during coding rounds; **read the problem carefully before coding**.
- Stay calm, explain your thought process step-by-step, and **don't rush the solution**.

9. Documentation & Notes

- Maintain **project notes, diagrams, and pseudocode** to help recall details during the interview.
- Highlight **your contributions and technical depth** for each project.

10. Stay Updated

- Keep track of **latest trends in ML/NLP, AI, and cloud technologies** relevant to your projects and the company domain.

“The harder you work for something, the greater you’ll feel when you achieve it.”

Reach Out :

Sanjay B 2022506124

B.Tech IT MIT-AU

Gmail: sanjayboopathy32@gmail.com

[LinkedIn](#)